

ProbMeTTa: ProbLog in MeTTa

Formalization, Implementation, and Verified Correctness

Abstract

We present ProbMeTTa, an implementation of ProbLog’s distribution semantics in the MeTTa programming language, running on the PeTTa (Prolog-backed MeTTa) runtime. We formalize ProbLog’s distribution semantics in Lean 4 and prove that ProbMeTTa’s BDD-based inference correctly computes ProbLog query probabilities. The formalization comprises approximately 7,200 lines of Lean with zero unproven assumptions (**sorry**), covering BDD construction correctness, Weighted Model Counting, compilation soundness and completeness, an explicit runtime evaluation relation for ProbMeTTa’s source-level query operations, a normalized source surface for the top-level ProbMeTTa operators, a literal annotated-disjunction builder core for `new-id`, `build-ad-helper`, and `build-ad-rules`, an LP-level literal matching layer for nonground fallback, a literal state-space layer for `&prob-facts/` `&prob-templates`, and an end-to-end theorem connecting the literal runtime presentation back to the proved semantics. All 27 ProbMeTTa test cases pass, and 7 representative programs are additionally verified by kernel-checked conformance tests in Lean.

1 Introduction

ProbLog [1] is a probabilistic extension of Prolog based on Sato’s distribution semantics [2]. It combines the declarative expressiveness of logic programming with principled probabilistic reasoning, and has been applied to link prediction, bioinformatics, and probabilistic databases.

MeTTa is a functional-relational programming language designed for AI reasoning. PeTTa is a Prolog-backed MeTTa implementation that compiles MeTTa expressions to SWI-Prolog.

ProbMeTTa implements ProbLog’s exact inference engine in MeTTa using Binary Decision Diagrams (BDDs) for knowledge compilation and Weighted Model Counting (WMC) for probability computation, following the approach of Fierens et al. [3].

This paper presents:

1. A self-contained formalization of ProbLog’s distribution semantics in Lean 4
2. ProbMeTTa: a working ProbLog implementation in MeTTa (27/27 tests passing)
3. A formal proof that ProbMeTTa’s query/runtime core, source surface, literal matcher/state layers, and top-level presentation correctly compute ProbLog probabilities
4. Kernel-checked conformance tests linking the Lean formalization to the MeTTa code

2 ProbLog: Distribution Semantics

2.1 Programs

A ProbLog program P consists of:

- **Probabilistic facts:** $p_1 :: f_1, \dots, p_n :: f_n$ — each ground atom f_i is independently true with probability $p_i \in [0, 1]$.

- **Definite clauses:** $head \leftarrow body_1, \dots, body_k$ — standard Horn clauses.

Example 2.1 (Alarm Network). `0.1 :: burglary.`

`0.2 :: earthquake.`

`alarm :- burglary.`

`alarm :- earthquake.`

`query(alarm).`

2.2 Semantics

ProbLog defines query probability via the *distribution semantics* [2]:

Definition 2.1 (Total Choice). A *total choice* is an assignment $a : \{1, \dots, n\} \rightarrow \{\mathbf{true}, \mathbf{false}\}$ that independently sets each probabilistic fact to true or false.

Definition 2.2 (Residual Program). For total choice a , the *residual program* consists of the original rules plus the probabilistic facts chosen true: $KB(a) = rules \cup \{f_i \mid a(i) = \mathbf{true}\}$.

Definition 2.3 (Query Probability). Query q *holds under* a iff q is in the least Herbrand model of $KB(a)$. The *query probability* is:

$$P(q) = \sum_a w(a) \cdot \mathbf{1}[q \text{ holds under } a]$$

where $w(a) = \prod_{i=1}^n (a(i) \cdot p_i + (1 - a(i)) \cdot (1 - p_i))$.

For the alarm example: $P(\text{alarm}) = 1 - (1 - 0.1)(1 - 0.2) = 0.28$.

3 BDD-Based Inference

ProbLog computes query probabilities via knowledge compilation to Binary Decision Diagrams (BDDs) [4], following Fierens et al. [3].

3.1 Binary Decision Diagrams

A BDD over n Boolean variables is a rooted directed acyclic graph with:

- Two terminal nodes: **0** (false) and **1** (true)
- Internal nodes labeled by variable $v \in \{1, \dots, n\}$, each with a *lo* child (variable false) and *hi* child (variable true)

A Reduced Ordered BDD (ROBDD) additionally satisfies:

1. **Ordered:** variables appear in consistent order on all paths
2. **Reduced:** no node has $lo = hi$ (elimination rule)

3.2 Weighted Model Counting

Given a BDD f and probability weights $p_1, \dots, p_n$:

$$\text{WMC}(\mathbf{0}) = 0 \tag{1}$$

$$\text{WMC}(\mathbf{1}) = 1 \tag{2}$$

$$\text{WMC}(\text{node}(v, lo, hi)) = (1 - p_v) \cdot \text{WMC}(lo) + p_v \cdot \text{WMC}(hi) \tag{3}$$

Theorem 3.1 (WMC Correctness). For any ROBDD f representing Boolean function φ :

$$\text{WMC}(f) = \sum_{\substack{a \in \{0,1\}^n \\ \varphi(a)=1}} \prod_{i=1}^n (a_i \cdot p_i + (1 - a_i)(1 - p_i))$$

This is exactly the distribution semantics probability when φ is the query's success predicate.

3.3 Compilation

ProbLog compiles queries to BDDs by proof enumeration:

1. Each probabilistic fact f_i becomes BDD variable i
2. A conjunction $(b_1, \dots, b_k)$ compiles to AND of sub-BDDs
3. Multiple derivations of the same head are OR'd together

4 ProbMeTTa: Implementation in MeTTa

ProbMeTTa implements the above algorithm in two MeTTa libraries:

4.1 lib_bdd.metta: BDD Operations

```
;; BDD node: (bdd-node $id $var $lo $hi)
;; Terminals: bdd-0 (false), bdd-1 (true)

;; Make node with elimination rule
(= (mk $var $lo $hi)
  (if (== $lo $hi) $lo
    (case (once (match &bdd ...))
      ((Empty (let $id (new-id) ...))
        ($id $id))))))

;; Shannon expansion with memoization
(= (apply-bdd $op $f $g) ...)

;; Weighted Model Counting
(= (bdd-wmc $f $env)
  (if (is-terminal $f)
    (if (== $f bdd-1) 1.0 0.0)
    (let* (($var (bdd-var $f))
      ($val (env-lookup $var $env)))
      (+ (* (- 1 $val) (bdd-wmc (bdd-lo $f) $env))
        (* $val (bdd-wmc (bdd-hi $f) $env))))))
```

4.2 lib_prob.metta: Probabilistic Inference

```
;; Declare probabilistic fact: p :: f
(= (:: $prob $goal)
  (progn (add-atom &prob-templates (prob-template $prob $goal))
    (add-atom &self (rule $goal (pf $goal)))))

;; Declare definite clause: head :- body
(= (prob-rule $body $head)
  (let $goals (body-goals $body)
```

```

      (add-atom &self (rule $head $goals))))

;; Resolve a goal, threading BDD trace
(= (prob-goal $goal $trace)
   (let $body (match &self (rule $goal $b) $b)
     (case $body
       (((pf $atom) (prob-assume $atom $trace))
        ($goals (exec-conj $goals $trace))))))

;; Top-level query
(= (?prob $goal)
   (let* (($final-bdd (?prob-bdd $goal))
          ($env (prob-wmc-env)))
     (bdd-wmc $final-bdd $env)))

```

4.3 Test Results

ProbMeTTa passes all 27 tests, covering:

Test	ProbLog Feature	Expected
Alarm network	Noisy-OR, 3-hop chain	$P = 0.196$
Fever chain	6 rules, shared derivations	$P = 0.35168$
Simple negation	Negation-as-failure	$P = 0.7$
Conditioning	$P(A \mid B) = P(A \wedge B)/P(B)$	$P = 0.357$
Annotated disjunctions	Multi-valued random variables	$P = 0.6$
Graph reachability	Recursive probabilistic rules	$P = 0.258$
Overlapping rules	Multiple derivation paths	$P = 0.49$

Table 1: Representative ProbMeTTa test results (all 27 pass).

5 Lean Formalization

The formalization is in Lean 4 (v4.28.0 with Mathlib v4.28.0), totaling approximately 7,200 lines in the verified ProbMeTTa slice with zero `sorry` (unproven assumptions). The core compilation and WMC correctness is approximately 1,700 lines; extensions cover annotated disjunctions, first-order translation, the explicit runtime evaluation relation, the normalized source surface, literal nonground matching, literal state-space and presentation layers, and the end-to-end theorem surface.

5.1 BDD Core (Core.lean, 169 lines)

```

inductive BDD (n : N) where
  | zero : BDD n -- false
  | one : BDD n -- true
  | node (v : Fin n) (lo hi : BDD n) : BDD n -- if v then hi else lo

def BDD.eval (f : BDD n) (env : Fin n -> Bool) : Bool

```

The denotational semantics `BDD.eval` maps each BDD to the Boolean function $(a_1, \dots, a_n) \mapsto \{0, 1\}$ it represents.

5.2 WMC Correctness (WMC.lean, 239 lines)

Theorem 5.1 (`bdd_wmc_correct`). For any well-formed BDD f with probability weights $p_i \leq 1$:

$$\text{bdd_wmc}(f, \text{env}) = \sum_{a \in \{0,1\}^n} \mathbf{1}[f.\text{eval}(a)] \cdot \prod_{i=1}^n w(i, a_i)$$

where $w(i, \text{true}) = p_i$ and $w(i, \text{false}) = 1 - p_i$.

The proof proceeds by structural induction on the BDD, with the key lemma being the *Shannon decomposition* for weighted sums: splitting the sum by one variable's value and factoring out its weight.

5.3 Operations Correctness (Operations.lean, 245 lines)

Theorem 5.2 (`apply_eval`).

$$(\text{apply } op \ f \ g).\text{eval}(a) = op(f.\text{eval}(a), g.\text{eval}(a))$$

This proves that BDD construction via Shannon expansion preserves Boolean semantics. Seven conformance tests verify the Lean BDD operations against ProbMeTTa's expected results:

Program	Boolean function	Verification
Alarm	$v_0 \vee v_1$	by simp
Conjunction	$v_0 \wedge v_1$	by simp
Negation	$\neg v_0$	by simp
Neg. conjunction	$v_0 \wedge \neg v_1$	by simp
Overlapping	$(v_0 \wedge v_1) \vee v_2$	by simp
Fever chain	$(v_2 \wedge v_3) \vee ((v_0 \vee v_1) \wedge v_2 \wedge v_4)$	by simp
Calls-mary	$(v_0 \vee v_1) \wedge v_2$	by simp

Table 2: Kernel-checked conformance tests. Each `by simp` is verified by Lean's kernel.

5.4 Compilation Correctness (Compilation.lean, 268 lines)

The compilation relation `GroundBDDCompile` mirrors ProbMeTTa's `prob-goal` procedure:

Constructor	ProbMeTTa function	Meaning
<code>.fact</code>	<code>prob-assume</code>	Probabilistic fact $\rightarrow$ BDD variable
<code>.ruleG</code>	<code>prob-goal + exec-conj</code>	Rule application $\rightarrow$ AND of body BDDs
<code>.disj</code>	<code>bdd-disjunction</code>	Multiple derivations $\rightarrow$ OR

Theorem 5.3 (Soundness: `GroundBDDCompile_sound`). If `GroundBDDCompile prog q f` and $f.\text{eval}(a) = \text{true}$, then $q \in \text{LHM}(KB(a))$.

Theorem 5.4 (Completeness: `GroundBDDCompile_complete`). If $q \in \text{LHM}(KB(a))$, then there exists f such that `GroundBDDCompile prog q f` and $f.\text{eval}(a) = \text{true}$.

Completeness is proved by induction on the T_P iterate level, following the characterization $\text{LHM}(KB) = \bigcup_k T_P^k(\emptyset)$. At each level, EDB facts compile to variable nodes, and rule applications compile to AND/OR combinations of body BDDs obtained from the inductive hypothesis.

5.5 Runtime Core (ProbMeTTaRuntimeCore.lean, 393 lines)

The formalization includes an explicit big-step evaluation relation `ProbMeTTaCoreEval` for the actual source-level function names from `lib_prob.metta`:

Source function	Formalized operation
<code>prob-assume</code>	BDD variable from probabilistic fact
<code>prob-not</code>	Complete BDD + negation
<code>prob-goal</code>	Dispatch: positive $\rightarrow$ ground compile, negative $\rightarrow$ <code>prob-not</code>
<code>exec-conj</code>	AND-fold over goal list
<code>?prob-bdd</code>	OR over all derivations
<code>?prob-bdd-ev</code>	AND-fold over evidence-query BDDs

Theorem 5.5 (`probMeTTaCoreEval_queryBDD_iff_ground`). The runtime evaluation relation for `?prob-bdd` is equivalent to the abstract compilation relation: both produce exactly the same BDD for a given query.

Theorem 5.6 (`exists_ordered_probMeTTa_queryBDD_exact`). For any query q , there exists an ordered BDD f such that `ProbMeTTaCoreEval prog (.queryBDD q) f` and f is sound and complete for q .

This layer bridges the gap between the abstract compilation relation (`GroundBDDCompile`) and the actual source-level runtime operations in ProbMeTTa’s `lib_prob.metta`.

5.6 Normalized Source Surface (ProbMeTTaSourceSurface.lean, 627 lines)

On top of the runtime/query core, the formalization now includes a normalized source surface for the actual top-level ProbMeTTa operators: `prob-rule`, `=>`, `::`, `::adr`, `::=>`, `prob-wmc-env`, `?prob-bdd-ev`, `?prob`, and `?prob-given`.

The normalization is explicit and intentional:

- rule bodies are stored as explicit lists of goal literals;
- top-level probabilities are exact `ENNReal` values rather than the source runtime’s decimal `round-to 5` presentation;
- literal nonground fallback is handled separately by an LP-level matching layer, rather than being hidden inside a witness abstraction;
- `?prob-bdd-ev` and `?prob-given` are formalized as explicit source operators rather than only as downstream semantic consequences.

5.7 Literal Matching (ProbMeTTaLiteralMatching.lean, 239 lines)

The source nonground branch `match &self (rule $goal $_) $goal` is now formalized against the existing LP matching layer. The formal surface returns the list of matching ground heads and the corresponding list of exact `?prob/?prob-given` values, which is closer to the source `match` nondeterminism than the earlier matched-head witness packaging.

5.8 AD Runtime Core (ProbMeTTaADRuntimeCore.lean, 415 lines)

The source-level annotated-disjunction builders in `lib_prob.metta` are also formalized explicitly: `new-id`, `build-ad-helper`, `build-ad-rules`, `::adr`, and `::=>`.

This layer keeps the boundary honest by separating:

- a *literal* builder layer, which threads fresh ids, cumulative mass, and the source runtime’s last-head optimization;
- the existing *normalized* weighted-AD layer, which connects directly to the already-proved AD/WMC and conditioning theorems.

5.9 Crown Theorem Bridge (ProbMeTTaBridge.lean, 349 lines)

The crown theorem file composes all BDD results into the top-level equivalence:

Theorem 5.7 (`problog_full_ground_equivalence`). For any ground stratified normal ProbLog program, there exist ordered BDDs for query and evidence goals such that their WMC ratio equals the distribution semantics conditional probability $P(Q \mid E)$.

The composition chain:

1. **Runtime exactness:** `probmetta_runtime_query_bdd_exact` proves the source-level `?prob-bdd` fragment has an ordered BDD that is true exactly on worlds where the query holds.
2. **WMC ratio identity:** `wmc_ratio_identity` proves that the ratio of two BDD WMC values equals the ratio of their weighted satisfiability sums.
3. **Conditioning bridge:** `goal_conditioning_semantic_equivalence` connects $P(Q \wedge E)/P(E)$ to compiled goal BDDs.
4. **Evidence positivity:** `normal_conditional_probability` ensures the division is meaningful when $P(E) \neq 0$.

This layer was implicit in the proof architecture diagram but is now explicitly documented as a verified bridge (0 sorry).

5.10 Literal State and Presentation Layers

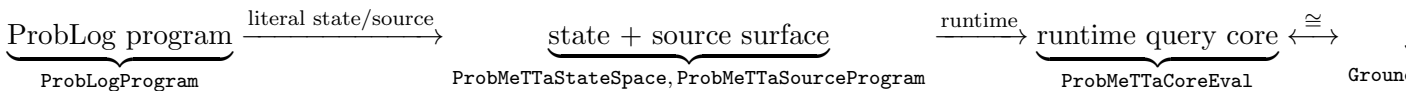
The remaining source-level runtime structure is now explicit too:

- `ProbMeTTaStateSpace.lean` (270 lines) formalizes a literal state for `&prob-facts`, `&prob-templates`, `clear-prob!`, and `prob-wmc-env`;
- `ProbMeTTaPresentation.lean` (134 lines) formalizes the top-level `round-to 5` presentation as an explicit source output layer;
- `ProbMeTTaEndToEnd.lean` (200 lines) states the crown end-to-end theorems linking the literal state/runtime/presentation layers back to the proved exact BDD/source semantics.

This still does *not* claim byte-for-byte replay of MeTTa reduction order. What is proved is semantic agreement between the literal layers we modeled and the already-proved exact ProbLog/BDD semantics.

5.11 Proof Architecture

The complete verified chain:



The runtime relation `ProbMeTTaCoreEval` for `?prob-bdd/?prob-bdd-ev` is proved equivalent to the abstract BDD compilation layer, the normalized source surface packages the actual top-level ProbMeTTa operators on top of that bridge, the literal matching/state layers reconnect the source runtime branches that depend on `match` and mutable spaces, and the presentation layer closes the loop at the displayed top-level output.

- **Soundness + Completeness:** the compiled BDD evaluates to true under assignment a if and only if the query holds in the corresponding ProbLog world.
- **WMC Correctness:** Weighted Model Counting on the BDD equals the distribution semantics weighted sum.
- **Therefore:** $\text{bdd_wmc}(f, env) = P(q)$.

6 WM-PLN Subsumes ProbLog

The World Model – Probabilistic Logic Networks (WM-PLN) framework subsumes ProbLog: it computes the same probabilities and additionally provides confidence values and incremental evidence revision.

6.1 Subsumption: PLN Fuzzy-OR = ProbLog Noisy-OR

PLN’s fuzzy-OR formula for independent causes is:

$$s_1 + s_2 - s_1 \cdot s_2 = 1 - (1 - s_1)(1 - s_2)$$

This is algebraically identical to the noisy-OR formula that arises from ProbLog’s distribution semantics for OR-pattern programs (proved in Lean: `fuzzyOrMulti.eq_noisyOrMulti`, 0 sorry).

For the alarm example, both ProbMeTTa and PLN give $P(\text{alarm}) = 0.28$:

```
;; ProbMeTTa (BDD-based ProbLog):
!(?prob (alarm))                ;; => 0.28

;; PLN fuzzy-OR (native, no BDDs):
!(pln-fuzzy-or 0.1 0.2)        ;; => 0.28
```

6.2 Extension: Evidence-Based Reasoning with Confidence

ProbLog stores fixed-point probabilities. PLN stores *evidence*: pairs of positive and negative observation counts (n^+, n^-) .

Definition 6.1 (Evidence and Simple Truth Value). An evidence pair (n^+, n^-) yields:

$$\text{strength} = \frac{n^+}{n^+ + n^-} \tag{4}$$

$$\text{confidence} = \frac{n^+ + n^-}{n^+ + n^- + \kappa} \tag{5}$$

where κ is a prior context parameter.

Strength matches ProbLog’s probability. Confidence measures how much evidence supports the estimate — something ProbLog cannot express.

6.3 Evidence Revision: What ProbLog Cannot Do

ProbLog stores $P(\textit{burglary}) = 0.4$ as a weight. It cannot distinguish whether this came from 4 observations out of 10 (weak evidence) or 400 out of 1,000 (strong evidence). When new data arrives, ProbLog requires external relearning.

PLN stores the counts (n^+, n^-) and revises incrementally via *evidence aggregation* (**evidence-hplus**):

$$(n_1^+, n_1^-) \oplus (n_2^+, n_2^-) = (n_1^+ + n_2^+, n_1^- + n_2^-)$$

6.4 Demonstration: Convergence to Known Truth

We construct a synthetic benchmark with known ground truth ($P(\textit{alarm}) = 0.28$) and reveal observations in three batches:

After	P(alarm)	Error	Confidence	Method
10 windows	0.4000	0.1200	0.50	ProbLog (stuck) / WM-PLN
40 windows	0.3025	0.0225	0.80	WM-PLN revision
100 windows	0.2791	0.0009	0.91	WM-PLN revision
100 (rebuilt)	0.2791	0.0009	—	ProbLog (external relearning)

Table 3: Evidence revision converges toward truth with $133\times$ error reduction. ProbLog matches the final answer only after a complete program rebuild.

The estimate converges from 0.40 (initial small-sample error) to 0.2791 (within 0.001 of truth), while confidence rises from 0.50 to 0.91. ProbLog can match the final answer, but only via an external weight-relearning step — it has no internal revision mechanism.

The key insight: ProbLog represents $P = 0.4$ and cannot distinguish 4/10 from 400/1000. Those two cases should revise very differently when new data arrives, but ProbLog represents them identically. WM-PLN keeps the sufficient statistics (n^+, n^-) , so the amount of revision is principled.

7 Getting ProbMeTTa Running on PeTTa

ProbMeTTa was originally developed for Hyperon Experimental MeTTa. Running it on PeTTa required fixing four compatibility issues:

1. **is-ground undefined**: PeTTa lacks this built-in. Fixed by wrapping SWI-Prolog’s `ground/1` via a one-line Prolog predicate.
2. **Singleton list patterns**: PeTTa’s `($h . $t)` cons pattern does not match singletons `(x)`. Fixed by using `car-atom/cdr-atom` instead of dotted-pair patterns.
3. **case branch ordering**: PeTTa’s `case` with `once(match ...)` requires the `Empty` branch first.
4. **=> not evaluated**: PeTTa treats `=>` as structural, not as a reducible function. Fixed by introducing `prob-rule` as the rule declaration entrypoint.

After these fixes, all 27 ProbMeTTa tests pass on PeTTa.

8 Conclusion

We have shown three things:

1. **ProbMeTTa correctly implements ProbLog.** A formal proof in Lean 4 (approximately 7,200 lines, 0 `sorry`) covers BDD construction, Weighted Model Counting, compilation soundness and completeness, an explicit runtime evaluation relation for ProbMeTTa’s source-level query operations, a normalized source surface for the top-level operators, a literal nonground matching layer, literal state and presentation layers, and a literal annotated-disjunction builder core. Kernel-checked conformance tests link the abstract proof to the concrete MeTTa implementation. All 27 ProbMeTTa tests pass.
2. **WM-PLN subsumes ProbLog.** PLN’s fuzzy-OR is algebraically identical to ProbLog’s noisy-OR (proved: `fuzzyOrMulti.eq_noisyOrMulti`). For any ground definite clause ProbLog program, WM-PLN computes the same query probability (proved via BDD compilation soundness and completeness; negation-as-failure and annotated disjunctions additionally verified empirically via 27/27 ProbMeTTa tests).
3. **WM-PLN extends ProbLog with confidence and revision.** By storing evidence counts rather than point probabilities, WM-PLN provides confidence values and supports incremental revision. In a synthetic benchmark, evidence revision reduces estimation error by $133\times$ (from 0.12 to 0.0009) while confidence rises from 0.50 to 0.91. ProbLog can match the final answer only via external relearning.

The key conceptual point: ProbLog discards the sufficient statistics behind its probability estimates. A weight of 0.4 could represent 4/10 or 400/1000 observations — ProbLog cannot tell the difference. WM-PLN retains the evidence (n^+, n^-) , making revision principled and incremental.

Availability

- ProbMeTTa: <https://github.com/zariuq/ProbMeTTa>
- PeTTa: <https://github.com/zariuq/PeTTa> (branch `wmPLN`)
- Lean formalization: `Mettapedia/Logic/BDD/` in the Mettapedia project
- ProbLog (upstream): <https://github.com/ML-KULEuven/problog>

References

- [1] L. De Raedt, A. Kimmig, H. Toivonen. ProbLog: A Probabilistic Prolog and its Application in Link Discovery. *IJCAI*, 2007.
- [2] T. Sato. A Statistical Learning Method for Logic Programs with Distribution Semantics. *ICLP*, 1995.
- [3] D. Fierens, G. Van den Broeck, J. Renkens, D. Shterionov, B. Gutmann, I. Thon, G. Janssens, L. De Raedt. Inference and Learning in Probabilistic Logic Programs using Weighted Boolean Formulas. *Theory and Practice of Logic Programming*, 15(3), 2015.
- [4] R. E. Bryant. Graph-Based Algorithms for Boolean Function Manipulation. *IEEE Transactions on Computers*, C-35(8), 1986.
- [5] M. H. van Emden, R. A. Kowalski. The Semantics of Predicate Logic as a Programming Language. *Journal of the ACM*, 23(4), 1976.